NWM: Negative Weight Mapping

A Non-Parametric Potential-Field Framework for Reinforcement Learning

CastermustOfficial
https://github.com/CastermustOfficial/NWM

July 20, 2026

Abstract

We present *Negative Weight Mapping* (NWM), a non-parametric reinforcement learning framework that represents an agent's experience as a persistent *potential field* over the observation space. Rather than training a value network or a policy by gradient descent, NWM accumulates a compact set of *centroids*—prototypical states that store per-action success and failure statistics—and selects actions by aggregating attractive (success) and repulsive (failure) forces from nearby centroids. Two mechanisms give the memory long-term stability: a *Dynamic Smart Lock* that protects high-confidence prototypes from being overwritten, and a *Fear & Greed* decision rule that rejects dangerous actions before maximizing reward. To act well under sparse reward, NWM additionally uses *temporally-correlated (sticky) exploration*, which repeats exploratory actions to build the sustained momentum that uniform noise cannot—and whose repeat probability can self-tune from the agent's own reward history. A further mechanism propagates value backwards along a graph of observed centroid transitions, letting outcomes stitch across episodes without disturbing the absolute score scale the decision rule depends on. We formalize the method and evaluate it against three baselines—a uniform random policy, tabular Q-learning with state aggregation, and a Deep Q-Network (DQN)—on three classic-control benchmarks spanning a difficulty gradient (CartPole, Acrobot, MountainCar), using one configuration shared across all tasks, 20 seeds, and paired per-seed significance tests. NWM decisively solves the sparse-reward MountainCar task, where every baseline including DQN fails to reach the goal even once (+64.7 return, $p < 10^{-3}$); it is competitive with DQN on dense CartPole, though the gap in its favour is not statistically established; and it trails DQN on the longer-horizon Acrobot. The MountainCar result reframes a failure mode often attributed to non-bootstrapping methods: there the obstacle is *exploration*, not temporal credit assignment, and a single task-agnostic change removes it. We also report a methodological finding that applies beyond this system. Earlier versions of this study used five seeds, a sample size at which the across-seed standard deviation of these benchmarks (130–140 return units) makes differences below roughly 120 units indistinguishable from noise; several conclusions we had drawn on that basis—including a mechanism trade-off we had shipped and documented—do not survive at 20 seeds. We report the corrected comparisons, including one in which fixing a bootstrapping bug in our own baselines reverses a result in DQN's favour. All code, seeds, and figures are released to make the study fully reproducible.

1 Introduction

Most modern reinforcement learning (RL) methods are *parametric*: a neural network compresses experience into weights updated by gradient descent [7, 9]. Parametric value learning is powerful but has well-known costs: it is sample-hungry, sensitive to hyperparameters, prone to catastrophic forgetting, and notoriously difficult to reproduce [4]. A complementary line of work is *non-parametric*

or *episodic* control, which stores experience explicitly and acts by similarity to past situations [2, 8]. Such methods can learn quickly from few interactions because a single informative experience is immediately usable, without waiting for it to be amortized into network weights.

NWM sits in this non-parametric family but takes an unusual inspiration: the *artificial potential field*, a classic idea from robot motion planning in which goals exert attractive forces and obstacles exert repulsive ones [5]. NWM treats past *successes* as attractors and past *failures* as repulsors in the observation space, so that action selection reduces to following the net force at the current state. The name *Negative Weight Mapping* refers to the explicit modelling of negative (repulsive) experience, which is used not merely to lower a value estimate but to actively veto risky actions.

This paper makes the original repository's method precise and subjects it to a controlled empirical study. Concretely, we contribute:

1. A formal description of the NWM memory, force model, and decision rule (Section 3), including the Dynamic Smart Lock and Fear & Greed mechanisms that were previously only described informally, and a temporally-correlated (sticky) exploration rule for sparse-reward tasks.

2. A reproducible benchmark harness comparing NWM against Random, tabular Q-learning, and DQN across three environments and 20 seeds, with a fixed greedy-evaluation protocol and aggregate statistics (Section 4).

3. An analysis of the results (Section 5) that identifies the regimes in which a potential-field method is competitive, together with an explicit discussion of its limitations (Section 6).

4. A methodological correction to our own earlier reporting (Section 4): at the five-seed sample size these benchmarks are routinely run with, the across-seed noise exceeds most of the effects being claimed. We show a mechanism ablation reversing sign between two five-seed samples, retract the conclusions we had drawn that way—including a trade-off we had shipped as a default—and re-report everything at 20 seeds with paired tests.

2 Related Work

Value-based RL. Q-learning [11] learns an action-value function by temporal-difference updates; with function approximation and experience replay it scales to high-dimensional inputs as DQN [6, 7]. We use both a tabular variant (with state aggregation) and DQN as baselines.

Episodic and non-parametric control. Model-Free Episodic Control [2] and Neural Episodic Control [8] store returns in a growing table keyed by (embedded) states and act by k-nearest-neighbour lookup, achieving strong early sample efficiency. NWM shares the store-and-retrieve philosophy but differs in three ways: (i) it aggregates experience into a bounded set of merged *centroids* rather than retaining individual transitions; (ii) it stores a *signed force* per action rather than a raw return estimate; and (iii) it separates *repulsion* into its own spatial-decay and veto pathway.

Potential fields. Artificial potential fields [5] are a staple of reactive control and motion planning. NWM imports the attractive/repulsive metaphor into RL by learning the field from returns rather than from a hand-specified geometry.

3 Method

3.1 Setup and notation

We consider a Markov decision process with observation space $\mathcal{S} \subseteq \mathbb{R}^d$ and a finite action set $\mathcal{A} = \{0, \dots, K-1\}$. Observations are z-score normalized online using running mean and variance maintained with Welford's algorithm [12]; we write $\tilde{x}$ for the normalized observation.

3.2 Persistent potential field

The memory is a bounded collection of centroids $\mathcal{M} = \{c_1, \dots, c_N\}$, $N \le N_{\max}$. Each centroid c_i stores

- a prototype location $\mu_i \in \mathbb{R}^d$ (a running mean of the normalized states merged into it) and a visit count n_i;
- per-action statistics $\{(s_{i,a}, n_{i,a}, q^2_{i,a})\}_a$ accumulating the sum, count, and sum of squares of the scores observed for action a; and
- a boolean *lock* flag ℓ_i.

The average score of action a at centroid i is $\bar{q}_{i,a} = s_{i,a}/n_{i,a}$.

Insertion and merging. A new experience $(\tilde{x}, a, \sigma)$ with score $\sigma \in [0, 1]$ (defined below) is routed to the nearest centroid $j = \arg\min_i \|\mu_i - \tilde{x}\|$. If $\|\mu_j - \tilde{x}\| < \tau_{\text{merge}}$ it is merged into c_j; otherwise a new centroid is created. Merging uses an exponential moving average with *progressive stiffness*: the mixing rate is $\alpha = 1/(n_j{+}1)$, and $\alpha = 1/(2n_j)$ once $n_j > 50$, so that well-established prototypes drift slowly. When N exceeds capacity, unlocked centroids are pruned by a value score $n_i\big((\bar{q}_i - 0.5)^2 + 0.1\big)$ that favours frequently-visited, strongly-signed prototypes; locked centroids are always retained.

3.3 Score assignment

Learning is episodic. At the end of an episode with return G, we compute a *percentile score* $p \in [0, 1]$ equal to the rank of G among all historical returns, so that scores are self-calibrating across environments with different reward scales. A temporal weight $w_t = 0.4 + 0.6\, t/T$ emphasizes later steps of a length-T episode, and the score written for step t is $\sigma_t = \text{clip}(p\, w_t, 0, 1)$.

Neutral-blended credit. Scaling p by w_t drives the early steps of a *good* episode toward 0—i.e. toward repulsion—even though those steps were not at fault. We therefore blend toward the neutral score 0.5 rather than toward 0,

$$\sigma_t = \text{clip}(0.5 + (p - 0.5)\, w_t,\ 0,\ 1)\,, \tag{1}$$

so that early steps of a good episode stay mildly attractive and early steps of a bad one stay mildly repulsive. Equation (1) is used throughout the reported benchmark.

Truncation-aware credit. The ramp w_t implicitly assumes the end of an episode explains its outcome, which is true for a true terminal event and false for a time-limit truncation. Given the `terminated`/`truncated` distinction we therefore set w_t per episode type: a poor episode ending in a genuine terminal event keeps the late-step ramp (the ending did cause the failure); a poor *truncated* episode—aimless until the clock ran out—receives flat mild credit, since no particular step is to blame; and a good truncated episode, which survived the entire limit, receives full flat credit instead of under-crediting the early steps that kept it alive.

Sign-aware quality gate. A *dynamic quality gate* caps $\sigma_t \leq 0.6$ (preventing attraction, which needs $\bar{q} > 0.6$, and locking, which needs $\bar{q} > 0.8$) for episodes that are not genuinely above trend. The natural threshold $\max(40,\, 1.25\,\bar{G}_{\text{recent}})$ is correct only for positive returns: when $\bar{G}_{\text{recent}} < 0$—as it is throughout training on Acrobot and MountainCar—the term $1.25\,\bar{G}_{\text{recent}}$ is *smaller* than $\bar{G}_{\text{recent}}$ and the constant 40 dominates, capping every episode at 0.6 and silently disabling both attraction and the Dynamic Smart Lock on exactly the tasks that need them. We therefore make the gate sign-aware, requiring an episode to beat its recent average by a fixed *relative* margin regardless of sign:

$$G \;\geq\; \bar{G}_{\text{recent}} + \tfrac{1}{4}\,\big|\bar{G}_{\text{recent}}\big|. \tag{2}$$

Only episodes clearing Eq. (2) can create locked, high-confidence memories.

3.4 Force model

Each centroid exposes a signed force per action,

$$f_{i,a} = \begin{cases} 1.5\,(\bar{q}_{i,a} - 0.5), & \bar{q}_{i,a} > 0.6 \quad \text{(attraction)} \\ 1.5\,(\bar{q}_{i,a} - 0.5), & \bar{q}_{i,a} < 0.4 \quad \text{(repulsion)} \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Given a query state $\tilde{x}$, let $\mathcal{N}_k$ be its k nearest centroids within a cutoff radius d_c, and $\delta_i = \|\mu_i - \tilde{x}\|$. The net force for action a is a confidence- and distance-weighted average,

$$F(a) = \frac{\sum_{i \in \mathcal{N}_k} f_{i,a}\;\omega_i\;\kappa_{i,a}\;\log(1+n_i)\;\beta_i}{\sum_{i \in \mathcal{N}_k} \omega_i}, \tag{4}$$

where the spatial weight ω_i decays as $1/(\delta_i + 0.1)$ for attraction but as the inverse square $1/(\delta_i^2 + 0.1)$ for repulsion—so danger signals are sharply local; $\kappa_{i,a} = 1/(1 + 2\,\mathrm{Var}_{i,a})$ down-weights high-variance (uncertain) prototypes; $\log(1+n_i)$ rewards evidence; and $\beta_i = \beta_{\text{lock}} > 1$ boosts locked prototypes.

3.5 Credit propagation on the centroid graph

Episode-level percentile credit is *horizon-blind*: every step of an episode inherits the same outcome, so a centroid can only learn from episodes that passed through it. Value never flows *between* episodes, which is precisely the advantage bootstrapped value learning retains on long horizons. We close part of that gap without touching the decision rule.

The field additionally records a *successor graph*: along each observed trajectory it stores transition counts $n(c, a \to c')$ for consecutive centroids, discarding self-loops, which carry no propagation information. At the end of each episode we run $S = 3$ sweeps of value propagation *in score space*,

$$V^{(k+1)}(c) = (1-\beta)\,\bar{q}_c \;+\; \beta\,\frac{\sum_a \sum_{c'} n(c, a \to c')\,V^{(k)}(c')}{\sum_a \sum_{c'} n(c, a \to c')}, \tag{5}$$

initialized at $V^{(0)}(c) = \bar{q}_c$, where $\bar{q}_c$ is the centroid's own recent score and $\beta \in [0, 1]$ is the propagation weight (centroids with no recorded successors keep $V(c) = \bar{q}_c$). The per-action force of Eq. (4) is then computed from a blended score

$$\hat{q}_{i,a} = (1 - \beta)\, \bar{q}_{i,a} + \beta\, \frac{\sum_{c'} n(c_i, a \to c')\, V(c')}{\sum_{c'} n(c_i, a \to c')} \tag{6}$$

in place of $\bar{q}_{i,a}$, falling back to $\bar{q}_{i,a}$ when action a has no recorded successors. A centroid whose own episodes ended badly can thus still learn that a given action leads into a good region—*trajectory stitching* across episodes.

The design constraint is deliberate. Because Eq. (5) operates in score space, $\hat{q}$ remains on the absolute $[0, 1]$ scale with 0.5 neutral, so the Fear & Greed thresholds, the safety veto, the lock criteria, and the confidence weights are all untouched. This matters: as Section 6 reports, earlier attempts to strengthen credit assignment by *replacing* the force formula (return-to-go percentiles, an advantage-style model) degraded performance sharply, because the decision rule is co-designed around absolute scores. Propagation succeeds where those failed by changing what is scored, not how scores are read. Setting $\beta = 0$ recovers the pure episode-credit method exactly.

3.6 Action selection: Fear & Greed

NWM first avoids danger, then seeks reward. With a risk tolerance $\rho = -0.5$ (tightened to $\rho = -1.5$ when a strong option exists, $\max_a F(a) > 0.8$), the safe set is $\mathcal{A}_{\text{safe}} = \{a : F(a) > \rho\}$, and the greedy action is $\arg\max_{a \in \mathcal{A}_{\text{safe}}} F(a)$ (falling back to the global argmax if every action is deemed unsafe). Exploration is ε-greedy with an *adaptive* rate that collapses to 0 once recent performance is high, and the agent explores by default in unfamiliar regions ($\min_i \delta_i > d_c$). The Dynamic Smart Lock sets ℓ_i=true once $n_i >$ `lock_min_visits` and $\bar{q}_i >$ `lock_min_score`. Algorithm 1 summarizes one training episode.

Temporally-correlated exploration. A uniform exploratory policy produces a zero-mean, rapidly-decorrelating action sequence—adequate for tasks solvable by local corrections, but pathological for tasks that require a *sustained, directed* sequence of actions to reach any rewarding state at all (e.g. pumping energy into an under-actuated system). We therefore allow *sticky* exploration: an exploratory step repeats the previous action with probability p_{rep} and otherwise samples uniformly,

$$a_t \;=\; \begin{cases} a_{t-1}, & \text{w.p. } p_{\text{rep}} \\ \text{Uniform}(\mathcal{A}), & \text{w.p. } 1 - p_{\text{rep}}, \end{cases} \qquad \text{(exploratory steps only).} \tag{7}$$

This injects temporal correlation—and thus the momentum-building action runs needed under sparse reward—without any task-specific knowledge; p_{rep}=0 recovers the original uniform exploration. Section 5 shows that this single change is what separates an exploration failure from a genuine credit-assignment failure on our sparse-reward task.

Rather than hand-tuning p_{rep}, it can also be set *adaptively*: while the recent returns are perfectly flat—the signature of a sparse-reward task whose goal has never been reached, since every episode then scores identically—p_{rep} is ramped up by 0.1 per episode (to at most 0.95), and the ramp stops as soon as returns vary. The trigger consumes only the agent's own reward history, so the mechanism remains fully task-agnostic; on tasks that give a learning signal from the first episodes it never fires.

Algorithm 1 NWM training episode

Input: field $\mathcal{M}$, environment, exploration rate ε, sticky probability p_{rep}, propagation weight β
Roll out the episode: at each step, with prob. ε explore (repeat a_{t-1} w.p. p_{rep}, else uniform), otherwise apply the Fear & Greed rule on Eq. (4) using the blended score Eq. (6); buffer $(\tilde{x}_t, a_t, r_t)$.
Compute return $G = \sum_t r_t$ and percentile score p.
gate $\leftarrow \left[\, G \geq \bar{G}_{\text{recent}} + \frac{1}{4}|\bar{G}_{\text{recent}}| \,\right]$ ▷ Eq. (2)
$w_t \leftarrow$ flat if the episode was truncated, else the ramp $0.4 + 0.6\, t/T$
for each buffered step t **do**
    $\sigma_t \leftarrow \text{clip}(0.5 + (p - 0.5)\, w_t, 0, 1)$
    if $\neg$ gate **then** $\sigma_t \leftarrow \min(\sigma_t, 0.6)$
    end if
    Insert/merge $(\tilde{x}_t, a_t, \sigma_t)$ into $\mathcal{M}$; update locks.
    Record the transition $(c_t, a_t) \rightarrow c_{t+1}$ in the successor graph.
end for
if $\beta > 0$ **then** run S sweeps of Eq. (5) over $\mathcal{M}$
end if
Update adaptive ε and p_{rep} from recent returns.

3.7 Complexity and implementation

With N centroids and dimension d, a query costs $O(Nd)$ for the nearest-neighbour scan. Crucially, the field is immutable during a rollout, so the stacked matrix of centroid locations is cached and reused across every action selection within an episode; merges and insertions maintain it incrementally in $O(d)$, and only the (rare) pruning step forces a rebuild. The implementation depends only on NumPy [3] and Gymnasium [10], uses a per-agent random generator for reproducibility, and is released with a strict-typed, tested codebase.

4 Experiments

Environments. We evaluate on three Gymnasium [10] classic-control tasks chosen to span a difficulty gradient: **CartPole-v1** (dense reward, short horizon), **Acrobot-v1** (shaped negative reward, longer horizon), and **MountainCar-v0** (sparse reward, requires momentum building). This spread tests not only whether NWM works but *where* it holds up.

Baselines. (i) **Random**: uniform policy, a lower bound. (ii) **Tabular Q-learning** with uniform state aggregation (α=0.1, γ=0.99, ε annealed $1 \rightarrow 0.02$). (iii) **DQN** [7]: a two-hidden-layer MLP (128×128), replay buffer, target network, Huber loss, Adam (10^{-3}), γ=0.99.

A correction to the value-based baselines. Earlier versions of this study reported both value-based baselines with a defect that we have since fixed, and we report it explicitly because it *reverses* one of our previous headline claims. Gymnasium distinguishes `terminated` (a true absorbing state) from `truncated` (the time limit expired). Our DQN and tabular Q-learning implementations treated both alike, zeroing the bootstrap target $\max_{a'} Q(s', a')$ on truncation. This is a well-known bug: it tells the learner that the world ends whenever the clock does, poisoning value estimates exactly on long-horizon tasks. Both baselines now bootstrap through truncations and only zero the target on true termination. The effect is large and it is *not* in our favour—on Acrobot, DQN improves

from -335.6 to -123.9 and becomes the strongest method on that task, overturning a lead we had previously claimed for NWM. All numbers reported below use the corrected baselines. Beating a handicapped baseline is not a result, and we would rather retract the claim than keep it.

Protocol. Each (environment, agent, seed) run trains for a fixed episode budget; every ten episodes we run a *greedy* evaluation of ten rollouts on seeds disjoint from training. We report the mean and standard deviation of the final greedy evaluation, the best evaluation reached, and the normalized area under the evaluation curve (AUC) as a sample-efficiency proxy.

How many seeds: a correction to our own protocol. Earlier versions of this study reported five seeds, and we now regard that protocol as inadequate—not as a matter of taste, but because it cannot support the claims that were made on it. The across-seed standard deviation of the final greedy evaluation is roughly 130–140 return units on both CartPole and Acrobot, which at n=5 gives a standard error near 60. Any difference smaller than roughly 120 units is therefore indistinguishable from noise at that sample size, and essentially every mechanism-level comparison we previously drew fell inside that band. We verified the consequence directly: a mechanism ablation measured on one set of five seeds reversed sign on another set of five (Section 5.1). Both readings were noise.

We therefore report n=20 seeds throughout, and we separate seeds by role so that no result is reported on seeds that influenced a choice: the shared configuration was selected on seeds $\{10, \ldots, 19\}$, the mechanism ablation of Section 5.1 was run on seeds $\{20, \ldots, 39\}$, and the headline comparison below uses the disjoint seeds $\{40, \ldots, 59\}$, which were touched exactly once. At n=20 the standard error falls to roughly 30, and we additionally report paired per-seed tests for the ablation, where pairing removes the across-seed variance that dominates these environments. We flag comparisons that remain inside the noise band rather than narrating them as findings.

Fairness: one shared configuration, and a held-out seed split. Two safeguards protect the comparison from favouring NWM. First, NWM uses a *single shared configuration on every environment* (`NWM_SHARED_CONFIG`: neutral-blended credit, sign-aware gate, adaptive stickiness, truncation-aware credit, sticky evaluation fallback, β=0.3, warmup 20, τ_{merge}=0.5), with no per-task overrides—exactly the constraint the DQN baseline operates under. Earlier versions of this study tuned NWM per environment while holding DQN fixed, which inflated the comparison. Second, as described above, the configuration was selected on seeds $\{10, \ldots, 19\}$ and the numbers reported below come from the disjoint seeds $\{40, \ldots, 59\}$, so the configuration was never tuned against the results we report. Following recommendations for reliable RL reporting [1, 4], all seeds and the full protocol are released. The complete study is reproduced by:

```
python -m benchmarks.run_benchmark -seeds $(seq 40 59)
```

Absolute values shift somewhat with library versions; the numbers below were produced with CPU PyTorch 2.6 and Gymnasium 1.2. NWM itself depends only on NumPy and is deterministic given a seed, so its numbers reproduce exactly; the value-based baselines inherit PyTorch's version-dependent numerics.

5 Results

Table 1 reports final greedy-evaluation scores (mean $\pm$ std over 20 seeds) and AUC. Figure 1 shows the learning curves and Figure 2 the final-performance comparison.

Table 1: Final greedy evaluation (mean ± std over 20 seeds) and normalized AUC. Best final-eval per environment in **bold**. Higher is better for all environments (Acrobot and MountainCar returns are negative). Bold marks the best *mean* only; Table 2 shows which of these gaps are statistically supported, and on CartPole and Acrobot they are not.

Environment	Metric	Random	TabularQ	DQN	NWM
CartPole-v1	Final eval	22.9 ± 2.2	139.9 ± 30.5	162.1 ± 97.0	**210.6 ± 105.9**
	AUC	22.1	114.4	109.3	135.1
Acrobot-v1	Final eval	-499.0 ± 2.6	-420.6 ± 45.9	**-216.6 ± 178.9**	-317.7 ± 145.8
	AUC	-498.5	-438.2	-355.2	-380.1
MountainCar-v0	Final eval	-200.0 ± 0.0	-200.0 ± 0.0	-200.0 ± 0.0	**-135.3 ± 25.8**
	AUC	-200.0	-200.0	-199.9	-142.6

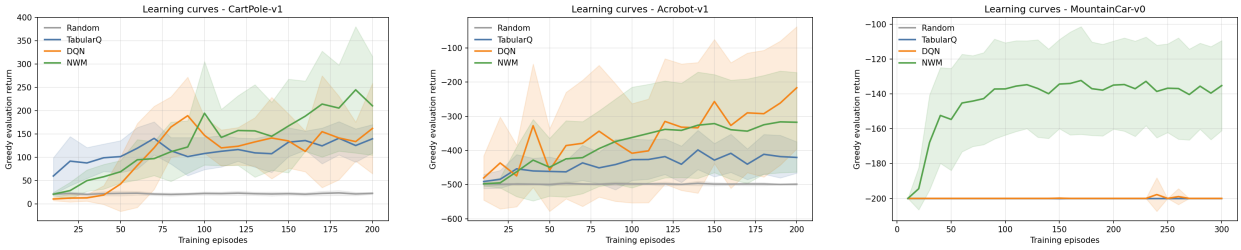

Figure 1: Greedy-evaluation learning curves (mean over 20 seeds, shaded ± std) for CartPole, Acrobot, and MountainCar.

Because every agent is run on the same seeds, we compare them *paired* per seed as well as in aggregate; Table 2 reports the paired differences with 95% confidence intervals. We lead with these rather than with the means of Table 1, because two of the three environments turn out not to support the conclusion their means suggest.

On the sparse-reward task, NWM is decisively the strongest method—and the failure was exploration, not credit assignment. MountainCar-v0 gives no reward until the goal is reached, and *every* baseline—Random, tabular Q, and DQN alike—finishes at exactly -200.0 ± 0.0: none reaches the goal once within the episode budget. NWM reaches -135.3 ± 25.8, a paired advantage of $+64.7$ $[+53.1, +76.3]$ on 18 of 20 seeds ($p = 0.0002$). This is the one result in the study that is unambiguous, and its cause is identifiable: sticky exploration builds the momentum needed to crest the hill, the field then forms around the successful trajectories. Because the only change is the exploratory action distribution—the scoring and force model are untouched—this isolates the obstacle. On MountainCar the barrier to a non-bootstrapping method is generating experience of the goal, not propagating value once it is seen, and a single task-agnostic exploration change removes it. The mechanism needs no tuning: the adaptive ramp of Section 3 sets p_{rep} from the agent's own reward history and is what the shared configuration uses.

On the dense task, NWM has the best mean but the gap to DQN is not established. On CartPole-v1 NWM attains 210.6 ± 105.9 against DQN's 162.1 ± 97.0, and also the better sample efficiency (AUC 135.1 vs. 109.3). It beats tabular Q-learning significantly ($+70.7$, $p = 0.012$). But the paired advantage over DQN, $+48.5$, has a confidence interval spanning zero $[-14.3, +111.3]$ and NWM wins on 14 of 20 seeds ($p = 0.064$). The defensible claim is that NWM is *competitive with* DQN on this task, not that it beats it; we previously stated the stronger claim on the strength of a five-seed run, and it does not survive.

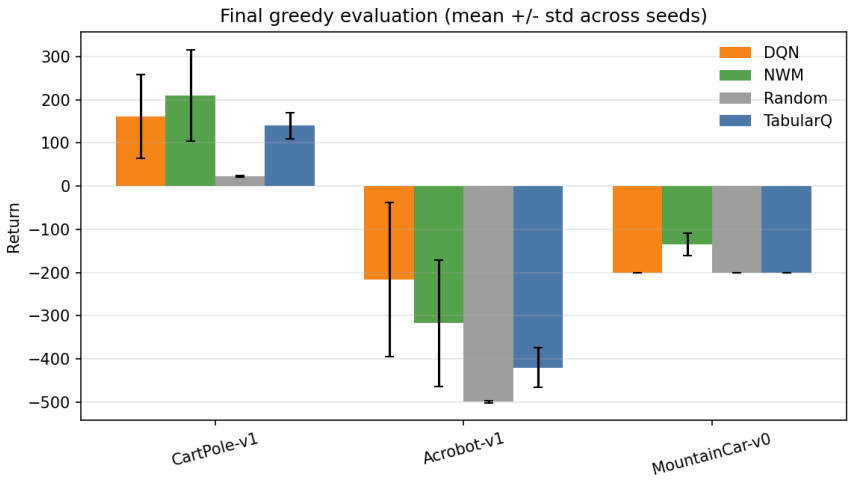

Figure 2: Final greedy evaluation per environment and agent (mean ± std over 20 seeds).

Table 2: Paired per-seed difference, NWM minus baseline, over the same 20 seeds. Positive favours NWM. "Wins" counts seeds on which NWM scores higher. p from a Wilcoxon signed-rank test.

Environment	Comparison	Δ (95% CI)	Wins	p
CartPole-v1	NWM − TabularQ	$+70.7$ $[+24.9, +116.4]$	13/20	0.012
	NWM − DQN	$+48.5$ $[-14.3, +111.3]$	14/20	0.064
Acrobot-v1	NWM − TabularQ	$+102.9$ $[+39.6, +166.3]$	13/20	0.011
	NWM − DQN	-101.1 $[-207.3, +5.2]$	5/20	0.071
MountainCar-v0	NWM − DQN	$+64.7$ $[+53.1, +76.3]$	18/20	**0.0002**

On the longer-horizon task, DQN is better, though the paired test is borderline. On Acrobot-v1 NWM reaches -317.7 ± 145.8 against DQN's -216.6 ± 178.9. The means understate how one-sided the per-seed picture is: NWM wins only 5 of 20 seeds, and the medians diverge much further than the means (-272.3 vs. -87.5), because DQN's mean is dragged down by a minority of collapsed runs while its typical run nearly solves the task. Both methods are strongly bimodal here—a run either learns or sits near the -500 floor—which inflates both standard deviations and leaves the paired difference at $p = 0.071$ despite the lopsided win count. We read this as DQN being clearly better in practice on Acrobot while cautioning that neither the size of the gap nor its formal significance is well determined at n=20. NWM does beat tabular Q-learning here ($+102.9$, $p = 0.011$). Longer horizons reward the temporal credit assignment of bootstrapped value learning that NWM's episodic percentile score, even with the propagation of Section 3.5, only partly approximates.

Summary. Of the three environments, one (MountainCar) yields a strong and well-supported result for NWM, one (CartPole) supports parity with DQN but not superiority, and one (Acrobot) favours DQN. Two of these three statements are weaker than the ones this paper made in earlier versions, and the change came entirely from increasing the seed count rather than from any change to the method.

Table 3: Paired ablation of credit propagation over 20 seeds. Δ is the mean per-seed difference (β=0.3 minus β=0); positive favours propagation. W/L counts seeds improved/worsened. Only MountainCar reaches significance.

Environment	β=0	β=0.3	Δ (paired)	W/L	Wilcoxon p
CartPole-v1	260.8	269.4	+8.6	10/10	0.93
Acrobot-v1	−276.6	−219.2	+57.5	12/8	0.37
MountainCar-v0	−159.7	−146.6	+13.1	13/4	**0.017**

5.1 Ablation: what credit propagation actually buys

Because credit propagation (Section 3.5) is the one mechanism in this release that changes what the field represents, we ablate it directly: β=0.3 against β=0, on the same 20 seeds ($\{20, \ldots, 39\}$, disjoint from both the selection and the reporting seeds), with every other setting fixed. Because the two configurations share seeds, we compare them *paired* per seed, which removes the across-seed variance that dominates these environments. Table 3 reports the result.

The honest reading is narrower than we first claimed. On Acrobot—the task propagation was designed for—the effect is large in the mean (+57.5) and in the right direction, but it is *not* statistically significant ($p = 0.37$, and 8 of 20 seeds get worse); Acrobot's returns are bimodal, a run either learns or sits near the −500 floor, and the mean is largely a count of how many runs escaped. On CartPole there is no effect at all (10/10 seeds split, $p = 0.93$). The only effect that survives a paired test is on **MountainCar** ($p = 0.017$, 13 of 20 seeds improved)—an environment for which propagation was not designed, and where we had previously claimed nothing.

We record this because our earlier five-seed measurements told a different and more flattering story: they showed a clear Acrobot gain bought at a clear CartPole cost, and we shipped and documented that trade-off. At n=20 neither half of it survives—the Acrobot gain is not significant and the CartPole cost does not exist. The trade-off was an artifact of reading a ±60 standard error as signal. We keep β=0.3 enabled, since it is neutral-to-positive on all three tasks and significantly positive on one, but the justification is not the one we previously gave.

6 Limitations

NWM has three principal limitations. First, its memory is instance-based, so query cost grows with the number of centroids and the method targets low-to-moderate dimensional observation spaces rather than raw pixels.

Second, credit assignment remains weaker than a bootstrapped Bellman backup, and this is still the residual gap on the longer-horizon Acrobot task. The propagation of Section 3.5 narrows it but is *shallow* in a specific sense: value flows only along transitions the agent has actually observed and only for a few sweeps, whereas DQN's backups generalize across the whole state space through the network, including to state-action pairs never visited consecutively. Propagation on a visited-trajectory graph is closer to model-free episodic stitching than to dynamic programming, and it inherits the graph's coverage—sparse regions propagate nothing. Consistent with this, the ablation of Section 5.1 cannot establish a significant Acrobot gain from propagation at n=20: the mean moves in the right direction, but 8 of 20 seeds still fail to escape the −500 floor, which is what a coverage-limited mechanism would predict. We stress, however, that the sparse-reward failure mode is genuinely resolved and was never a credit-assignment problem: on MountainCar the obstacle was exploration, removed by sticky exploration.

Third, the force model and its constants (thresholds, decays, lock criteria, the propagation weight β, and the sticky probability p_{rep}) are hand-designed. While robust across the tested tasks, they are not learned; β was chosen by a coarse sweep whose intermediate values we would no longer claim to distinguish, given the noise levels of Section 4, and p_{rep} trades reactivity for momentum—though it need not be set by hand, since the adaptive ramp of Section 3 tunes it from the agent's own reward history and is what the shared configuration uses. We view NWM as a transparent, reproducible baseline for non-parametric control rather than a state-of-the-art method, and a useful component for hybrid systems.

Negative results. We also report what did *not* work, since it sharpens the picture of where the residual Acrobot gap comes from. We implemented (i) *return-to-go percentile credit*, scoring each step by the percentile rank of its discounted return-to-go G_t against a rolling history, with truncated episodes bootstrapping their tail from the running mean per-step reward; and (ii) an *advantage force model*, replacing the absolute per-centroid force with each action's score advantage over the *other* actions observed at the same centroid (an argmax-style local comparison). Both were ablated on the protocol of Section 4: (i) was neutral on MountainCar and clearly worse on Acrobot (-378 to -400 vs. -247 for the shipped configuration over three seeds), because absolute per-step scores conflate distance-from-goal with action quality, turning the early states of good episodes repulsive; (ii) was worse still (-375 to -443), including when paired with the proven episode-level credit. The lesson is structural: NWM's decision rule—the Fear & Greed thresholds, the safety veto, the confidence weights—is co-designed around absolute scores with 0.5 as the neutral point, and swapping the force formula without redesigning the rule breaks the system. Closing the long-horizon gap appears to require joint redesign, not a drop-in bootstrapping patch; neither variant ships in the released code.

This diagnosis is what made the propagation of Section 3.5 work where those attempts failed: it strengthens credit while leaving the score scale—and therefore the decision rule—invariant. Three further mechanisms were implemented, measured, and *not* enabled in the shared configuration, and we report them for the same reason:

- *Recency-weighted scores* (`score_ema`): replacing all-time per-action sums with exponential moving averages, motivated by the non-stationarity of percentile scores (a 0.9 at episode 10 does not mean what a 0.9 at episode 190 means). It helps long-horizon Acrobot markedly ($-277.6 \to -201.6$ at α=0.3) but costs both CartPole and MountainCar, so it ships off.
- *Percentile-based exploration collapse* (`relative_explore`): replacing the CartPole-scaled absolute thresholds 450/300 with percentiles of the agent's own return history. It is principled—the constants are meaningless outside CartPole's reward scale—but measurably worse on Acrobot (-369.7) in the shared configuration. It ships off, available for custom setups.
- *Stagnation revival* (`stagnation_revival`): snapshotting the field on new bests and restoring it after a sustained collapse, targeting the bimodal across-seed distributions in which individual runs get trapped. On the selection seeds it did not improve the benchmark means (CartPole $298.2 \to 273.1$; Acrobot $-233.1 \to -265.4$), so despite its intuitive appeal it ships off.

We consider a mechanism's measured effect, not its plausibility, the criterion for shipping it. Two caveats apply to this list. First, all three were measured under the earlier five-seed protocol, so by the argument of Section 4 the individual figures should be read as indicative only; what we retain is the decision (none is enabled by default), not the precise magnitudes. Second, "did not help" at that sample size is a statement about our evidence rather than about the mechanisms, and we would not treat any of the three as refuted—`score_ema` in particular targets a real defect, the non-stationarity of percentile scores, and deserves a properly powered re-evaluation.

7 Conclusion

We formalized NWM, a potential-field, non-parametric RL framework, and evaluated it against Random, tabular Q-learning, and DQN on three classic-control tasks with a reproducible, seed-controlled protocol, using a single shared configuration and a held-out seed split so that neither per-task tuning nor a handicapped baseline flatters the method. The study clarifies the operating regime of instance-based potential-field control and provides an honest, open baseline: NWM is competitive on dense short-horizon control, decisively better on the sparse-reward task where exploration rather than credit assignment is the barrier, and still behind a correctly implemented DQN on the longest-horizon task.

Credit propagation on the centroid graph (Section 3.5) shows that some of the long-horizon gap is recoverable without abandoning the non-parametric design, provided the propagation is expressed in the score space the decision rule already speaks. The natural next steps follow from where it falls short: propagation over a *learned* similarity structure rather than only over observed transitions, so that value generalizes into unvisited regions; learned embeddings for higher-dimensional inputs; and hybrid schemes that use the repulsive field as an interpretable safety layer around a parametric policy.

Reproducibility Statement

The library, benchmark harness, exact seeds, hyperparameters, and the scripts that regenerate every table and figure in this paper are released at `https://github.com/CastermustOfficial/NWM`. Running the benchmark and `paper/make_paper_assets.py` regenerates `tables/results_table.tex` and the figures used above.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[2] Charles Blundell, Benigno Uria, Alexander Pritzel, Yazhe Li, Avraham Ruderman, Joel Z Leibo, Jack Rae, Daan Wierstra, and Demis Hassabis. Model-free episodic control. In *arXiv preprint arXiv:1606.04460*, 2016.

[3] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

[4] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

[5] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986.

[6] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3):293–321, 1992.

[7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[8] Alexander Pritzel, Benigno Uria, Sriram Srinivasan, Adrià Puigdomènech Badia, Oriol Vinyals, Demis Hassabis, Daan Wierstra, and Charles Blundell. Neural episodic control. In *International Conference on Machine Learning (ICML)*, pages 2827–2836, 2017.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[11] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, 1992.

[12] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.